

Project Presentation of CO-OP Project at Industry (Module-2) (22CS421)

On

**Scalable and Reliable Backend Data Integration Using Message-Driven
Distributed Architectures**

Submitted By:

**Jai Chopra
2210990427**

Supervised By:

**Dr.Preeti Saini
Assistant Professor**

**Department of Computer Science and Engineering,
Chitkara University, Punjab**

- 1 Introduction & Problem Statement
- 2 Research Objectives & Gap
- 3 System Architecture & Methodology
- 4 Experimental Setup
- 5 Results: Throughput, Latency & Scalability
- 6 Comparison with Synchronous REST
- 7 Conclusion & Future Work

The Problem with Synchronous Architecture

Tight Coupling

Services that directly call each other become structurally coupled — a failure in one cascades to all dependents.

Blocked Threads

Services waiting for responses hold compute resources hostage. Throughput drops as more threads become blocked.

Cascade Failures

A slow service degrades all upstream callers, turning a localized slowdown into a system-wide outage.

Scale Ceiling

A conventional Spring Boot REST service plateaued at just 4,500 req/sec — far below modern enterprise demands.

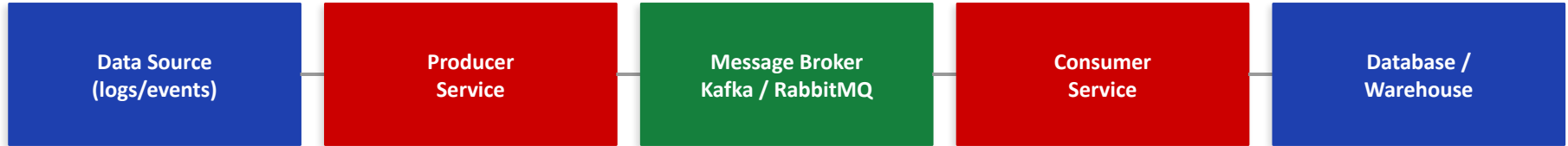
Research Objectives

1. Evaluate publish-subscribe vs. producer-consumer models in a realistic multi-service environment
2. Measure throughput, latency & scaling of Kafka vs. RabbitMQ under identical conditions
3. Quantify the improvement of async messaging over synchronous REST
4. Provide practical selection criteria for engineers choosing between the two systems

Research Gap

- ▶ Most studies benchmark only one broker — direct Kafka vs. RabbitMQ comparisons at variable cluster sizes are rare
- ▶ Almost no studies include a synchronous REST baseline, making it impossible to quantify real-world improvement
- ▶ Existing tests focus on raw speed, not full backend integration workflows
- ▶ Cloud-managed service studies offer limited tuning depth for demanding production workloads

Message-Driven Architecture Overview



Processing Flow

- 1 Generate synthetic JSON event data (user activity, system events, transactions)
- 2 Producer publishes message to broker topic/queue
- 3 Broker stores message durably (replication factor 3 for Kafka)
- 4 Consumer retrieves and processes message asynchronously
- 5 On failure: retry up to 3× with exponential backoff
- 6 Processed data written to PostgreSQL database

Environment & Configuration

NODES

5 Ubuntu 22.04 VMs

HARDWARE

4 vCPU · 8 GB RAM

NETWORK

1 Gbps private

KAFKA

v3.5.0 + ZooKeeper

RABBITMQ

v3.12.x (AMQP)

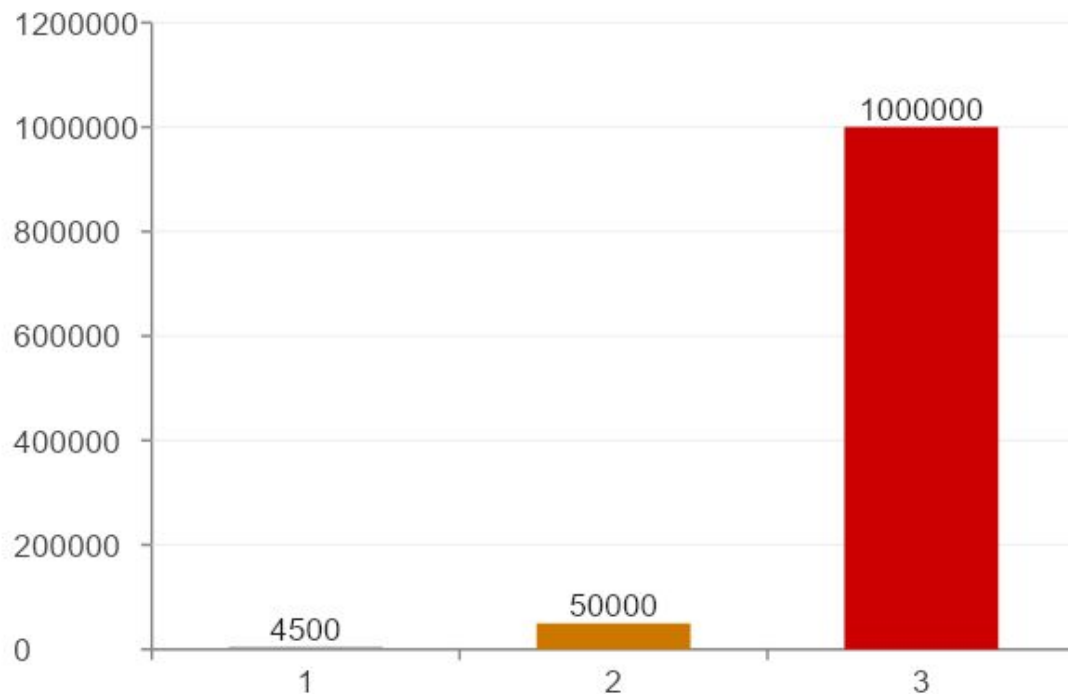
MONITORING

Prometheus + Grafana

Key Experimental Parameters

Parameter	Value	Parameter	Value
Message Size	1 KB avg (JSON)	Node Configs Tested	1, 3, 5 nodes
Producers / Consumers	2 producers · 3 consumers	Kafka Partitions (5-node)	12 per topic
Replication Factor	3 (survives 2 node failures)	RabbitMQ Prefetch	250
Acknowledgment	Manual ACK (at-least-once)	Test Duration per Run	5 minutes

Peak Throughput — Kafka vs RabbitMQ vs REST



Apache Kafka

1,000,000

msg/sec

RabbitMQ

50,000

msg/sec

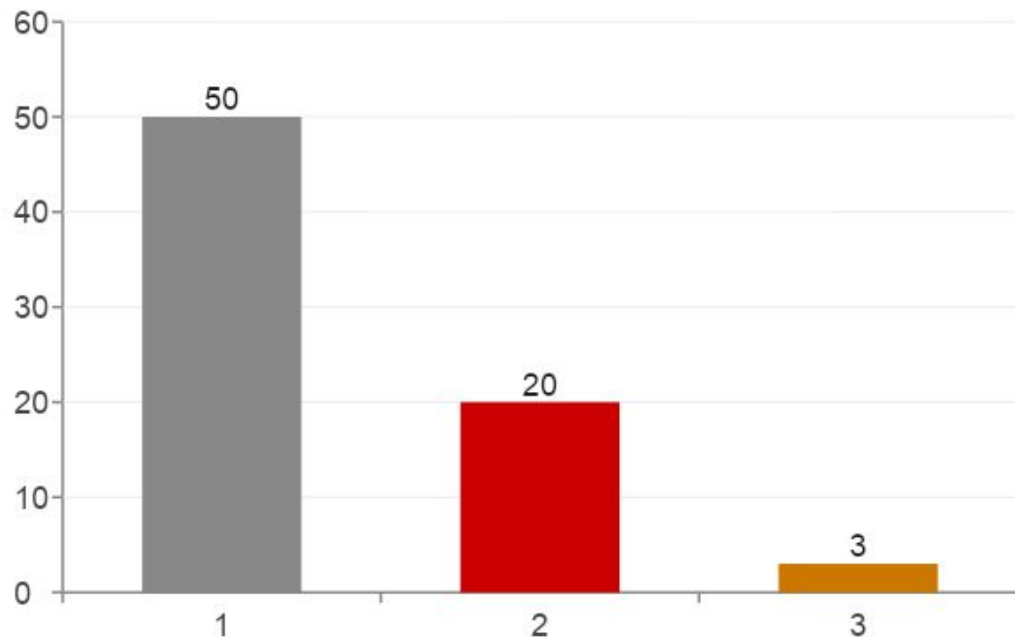
REST Baseline

4,500

req/sec

Kafka processes 222× more messages/sec than REST on identical hardware

End-to-End Delivery Latency (Producer → Consumer ACK)



RabbitMQ — 3 ms

Routes each message individually through memory. Best for fraud detection, live auctions, real-time monitoring.

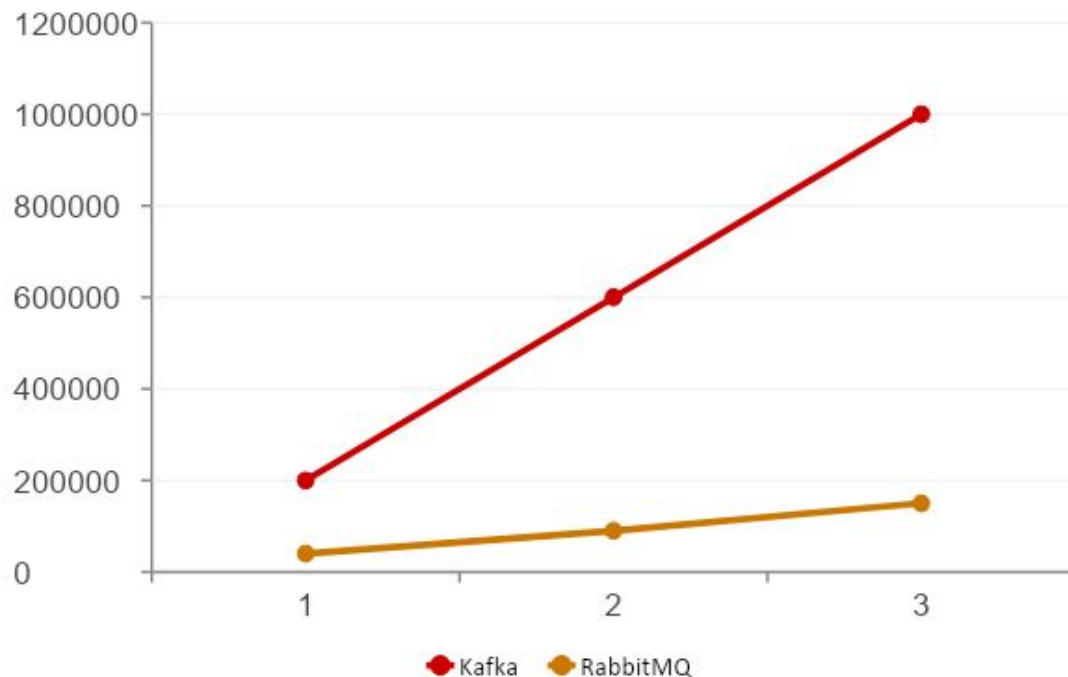
Kafka — 20 ms

Batches messages before writing to achieve high throughput. Best for log collection, clickstream analytics, ETL pipelines.

REST — 50 ms

Thread blocking under load causes compounding delays. Not suitable for high-volume inter-service communication.

Throughput vs. Number of Nodes



Nodes	Kafka (msg/s)	RabbitMQ (msg/s)	Kafka S(n)
1	200,000	40,000	1.0 (baseline)
3	600,000	90,000	3.0
5	1,000,000	150,000	5.0 (ideal linear)

Key Insight

Kafka achieved a perfect scaling factor of 5.0 across 5 nodes — each node takes on more partitions independently, with zero cross-node synchronization cost.

RabbitMQ scaling factor of 3.75 shows diminishing returns past 3 nodes due to exchange/queue binding overhead.

Async Messaging vs. Synchronous REST

System	Max Throughput	Min Latency	vs REST Baseline
REST (Synchronous Baseline)	4,500 msg/sec	~50 ms	1× (reference)
RabbitMQ	50,000 msg/sec	3 ms	11× throughput · 17× lower latency
Apache Kafka (5 nodes)	1,000,000 msg/sec	20 ms	222× throughput · 2.5× lower latency

Key Takeaways

- A 222× throughput gap cannot be bridged by hardware tuning — it reflects two fundamentally different communication models.
- Kafka for >50,000 events/sec workloads (log pipelines, clickstream, ETL). RabbitMQ when latency must be under 5 ms (fraud alerts, auctions).
- If still using synchronous REST for high-volume inter-service communication, the performance ceiling is simply too low for modern workloads.

Conclusion

- Async message brokers dramatically outperform synchronous REST — Kafka at 222× and RabbitMQ at 11× throughput improvement
- Kafka achieved a near-perfect linear scaling factor of 5.0 across 5 nodes
- RabbitMQ delivers 3 ms latency, 6.7× better than Kafka, ideal for time-critical workloads
- Partition count directly controls Kafka parallelism — plan partitions relative to cluster size
- The choice between Kafka and RabbitMQ depends entirely on workload characteristics, not one-size-fits-all

Future Work

ML-Driven Message Routing

A reinforcement learning model at the broker layer, watching queue depths and consumer speeds in real-time to route smarter than static partitioning

Hybrid Sync-Async Systems

Synchronous API surface for users, async messaging between internal services — measure the cost and break-even point of the translation layer

Real-World Traffic Validation

Reproduce findings with production traffic patterns including unpredictable spikes, mixed hardware, and network congestion

Thank You